FIFA World Cup 2026 Player Performance — Classifier Comparison

Course: M.Tech (AIML/DSE) — Machine Learning · **Assignment 2 Name / BITS ID:** `Badhmanaban M` / `2025AC05386`

a. Problem statement

Given a single match's performance record for one player — what he did on the pitch over those 90 minutes, plus his physical attributes — predict which of the four **positions** he plays: Goalkeeper, Defender, Midfielder or Forward. This is a **multi-class** classification problem over four classes.

The operational framing is role inference from telemetry. Match event feeds (passes, tackles, distance covered, sprint distance) arrive reliably; the position label is squad metadata that is frequently missing, stale, or wrong for players deployed out of their nominal role. A model that recovers position from behaviour lets a scouting or analytics pipeline (a) fill gaps in third-party feeds, and (b) flag the interesting disagreements — a "Midfielder" whose match profile scores as a Defender is exactly the tactical-role-change signal an analyst wants surfaced.

b. Dataset description

Property	Value
Source	FIFA World Cup 2026 — Player Performance (Kaggle)
Instances	**54,600** raw player-match rows (requirement: ≥ 500)
Features	**53** after cleaning (requirement: ≥ 12) — 48 numeric, 5 categorical
Target	`position` — Goalkeeper / Defender / Midfielder / Forward
Class balance	Defender 34.62% · Midfielder 30.77% · Forward 23.08% · Goalkeeper 11.54%
Missing values	**0** — no imputation was actually exercised, though the pipeline includes it
Rows used	15,000, stratified subsample (see *Why subsample* below)
Train / test split	11,187 / 3,813 rows — `GroupShuffleSplit` on `player_id`, `random_state=42`

The split is grouped, not stratified — and that is the important decision

The grain of this table is **player-match**, not player: 1,248 players contribute roughly 44 rows each. Several columns are constant within a player (`age`, `height_cm`, `weight_kg`, `market_value_eur`, `preferred_foot`). Under a plain `train_test_split`, the same player lands on both sides, those columns become an identity fingerprint, and the model is scored on players it has already memorised. The test number then measures recall of the training set, not generalisation.

This is not hypothetical. Measured on this data:

Target	Random row split	Grouped split (`player_id`)	Majority baseline
`preferred_foot`	0.846	**0.712**	0.745
`position`	0.950	**0.944**	0.346

`preferred_foot` looks like a real 0.85-accuracy result under a random split. Under a grouped split it falls **below the majority-class baseline** — the entire apparent skill was identity memorisation. `position` barely moves, which says its signal is genuinely per-match rather than per-player, and is why it is a sound choice of target here. Both numbers are reproduced by `scripts/verify.py`.

`StratifiedGroupKFold` is used for cross-validation for the same reason; ordinary `StratifiedKFold` would report the same inflated number and be worthless as a check.

Columns dropped, and why

Three distinct categories — see `scripts/train_fifa.sh`, which is the executable version of this list.

1. **Identity and fixture metadata** (11 cols) — `player_name`, `match_id`, `club_name`, `team`, `nationality`, `stadium`, `city`, `market_value_eur`, … No predictive content about *position*, and several uniquely fingerprint a player. `player_id` is held out separately as the grouping key and is never a feature.
2. **Tournament-level rollups** (5 cols) — `total_goals_tournament`, `total_minutes_tournament`, `tournament_rating`, `player_of_match_awards`, `total_assists_tournament`. These aggregate the whole competition, *including matches that fall in the test split*. Using them to predict a single match is target leakage across time.
3. **Team-match outcome** (4 cols) — `match_result`, `goals_team`, `goals_opponent`, `tournament_stage`. Properties of the fixture, not the player. Verified to carry no signal at all: a Random Forest predicting `match_result` from everything else scores **0.380** against a **0.369** majority baseline. The generator sampled these independently of the rest of the table, which is a useful reminder that this is synthetic data.

Preprocessing

Every model is wrapped in a single `sklearn.Pipeline`, so imputation and encoding are fitted on the training fold only and travel with the serialised model. The Streamlit app therefore cannot apply a different transform at inference than was used at training — the class of bug that produces a great notebook and a broken deployment.

- **Numeric** (48): median imputation → `StandardScaler`. Scaling is what makes kNN (Euclidean distance), logistic regression (convergence) and the RBF SVM (kernel width) work at all; it is a no-op for the tree-based models.
- **Categorical** (5: `preferred_foot`, `yellow_cards`, `red_cards`, `clean_sheet`, `penalty_saves`): mode imputation → `OneHotEncoder(handle_unknown="ignore")`, so a level present in an uploaded CSV but unseen in training encodes to all-zeros instead of raising at inference.
- **Counts are kept numeric.** The default heuristic sends any integer column with ≤ 10 distinct values to the one-hot branch — correct for an integer-coded month, wrong for `tackles` or `goals`, where the ordering is real. Forcing the 18 count columns back to numeric moved Naive Bayes from **0.489 → 0.706** accuracy (AUC 0.796 → 0.947) and was the single largest modelling change in this project. A Gaussian fitted to a 0/1 dummy is a poor density model.

Why subsample to 15,000 rows

`SVC` scales between $O(n^2)$ and $O(n^3)$ in training samples, and `probability=True` wraps it in an internal 5-fold Platt calibration. The RBF fit — not the data — is the binding constraint on end-to-end runtime. Sampling is stratified on `position` so the models see the population class prior. The full 54,600 rows are still 3.6× the assignment minimum after the cut.

---

c. GitHub repository link

- **Repository:** https://github.com/hardkidbadhu1/ml-assignment2
- **Live Streamlit app:** https://badhu-ml-assignment2.streamlit.app/

```
project-folder/
├── app.py                      Streamlit UI
├── ml_pipeline.py              shared preprocessing + metric code (train and serve)
├── requirements.txt            pinned runtime dependencies (what Streamlit Cloud installs)
├── requirements-dev.txt        build-time only — PDF rendering
├── runtime.txt                 Python version for Streamlit Cloud
├── README.md
├── test_data.csv               held-out split used by the app (2,000 rows)
├── data/                       raw dataset (gitignored — 17 MB)
├── model/
│   ├── train.py                training + evaluation entry point
│   └── artifacts/              *.joblib pipelines + metadata.json
├── reports/
│   ├── metrics.csv / metrics.md            generated metric tables
│   └── ML_Assignment2_...pdf               submission PDF, rendered from this README
└── scripts/
    ├── train_fifa.sh           the exact, documented training invocation
    ├── lab_run.sh              one-shot run for the BITS Virtual Lab (AWS VM)
    ├── verify.py               post-training leakage / artifact / deploy checks
    └── make_pdf.py             README.md -> submission PDF
```

Reproduce:

```
pip install -r requirements.txt
bash scripts/train_fifa.sh     # ~25 s
python scripts/verify.py       # 28 checks; exits non-zero on failure
streamlit run app.py

# Submission PDF (build-time deps only; see the note below)
pip install -r requirements-dev.txt
python scripts/make_pdf.py
```

The PDF is rendered *from this README*, so the two cannot drift. WeasyPrint is kept out of `requirements.txt` on purpose: it links against system Cairo/Pango, which Streamlit Cloud's build container does not have, so listing it there would break the deploy for a dependency the running app never imports.

Retrain before you deploy. The committed artifacts were fitted under Python 3.10 / scikit-learn 1.7.2. A `Pipeline` unpickled under a different scikit-learn version than it was fitted on is the most common Streamlit Cloud failure — it either raises `InconsistentVersionWarning` or, worse, silently mis-predicts. Re-run `scripts/train_fifa.sh` locally, then copy the version block it prints at the end into `requirements.txt` and set the matching `python-3.x` in `runtime.txt` and in the Streamlit Cloud app's *Advanced settings*. The app also surfaces a banner in the sidebar if the runtime's scikit-learn differs from the one recorded in `metadata.json`.

d. Models used

All six models are trained on the same dataset, the same grouped split, and the same preprocessing pipeline, so the comparison isolates the effect of the learning algorithm and nothing else.

Model	Key hyperparameters	Why included
Logistic Regression	`max_iter=2000`	Linear baseline; calibrated probabilities
Decision Tree	`min_samples_leaf=5`	Non-linear, interpretable, high variance
kNN	`k=15`, distance-weighted	Instance-based; depends entirely on the scaler
Naive Bayes (Gaussian)	defaults	Generative; strong independence assumption
Random Forest	`n_estimators=200`, `min_samples_leaf=5`	Bagging ensemble — variance reduction
SVM (RBF)	`probability=True`	Max-margin with kernel; the sixth model

`min_samples_leaf=5` on the forest rather than the more usual 2: a fully grown 300-tree forest pickles to 46 MB, which is awkward to commit and wasteful to hold in Streamlit Cloud's memory. The trade costs 0.003 accuracy for a 2.4× smaller artifact.

Comparison table

Held-out test set (3,813 rows, 312 players never seen in training). Precision / Recall / F1 are **macro**-averaged, AUC is **one-vs-rest macro**.

Produced on the BITS Virtual Lab VM (Rocky Linux 9.5, Python 3.12.7, scikit-learn 1.7.2) — the same run as the submitted screenshot, and the same artifacts committed under `model/artifacts/`. Fit times are from that 8-vCPU instance; the metrics themselves reproduce exactly on other hardware, since every model is seeded with `random_state=42`.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	**0.9580**	**0.9972**	**0.9608**	**0.9573**	**0.9589**	**0.9416**
Decision Tree	0.8985	0.9602	0.9041	0.8973	0.9002	0.8587
kNN	0.8893	0.9829	0.9001	0.8839	0.8911	0.8457
Naive Bayes (Gaussian)	0.7060	0.9465	0.7197	0.7713	0.7011	0.6310
Random Forest (Ensemble)	0.9441	0.9948	0.9486	0.9439	0.9461	0.9223
SVM (RBF)	0.9533	0.9961	0.9567	0.9502	0.9533	0.9349

Diagnostics

5-fold `StratifiedGroupKFold` on the training split only.

Model	Train Acc	Test Acc	Gap	CV Acc (mean ± sd)	Fit time (s)
Logistic Regression	0.9621	0.9580	**0.004**	0.9551 ± 0.0066	1.44
Decision Tree	0.9634	0.8985	**0.065**	0.8999 ± 0.0021	0.46
kNN	1.0000	0.8893	**0.111**	0.8866 ± 0.0101	0.20
Naive Bayes (Gaussian)	0.7293	0.7060	0.023	0.7274 ± 0.0159	0.19
Random Forest (Ensemble)	0.9790	0.9441	**0.035**	0.9445 ± 0.0064	3.69
SVM (RBF)	0.9631	0.9533	0.010	0.9518 ± 0.0075	15.72

Observations

ML Model Name	Observation about model performance
Logistic Regression	Best model on every one of the six metrics, and it is the *simplest* one — which is the finding, not an accident. The engineered composite columns (`offensive_contribution`, `defensive_contribution`, `creativity_score`) are close to linear discriminants for role, so the four classes are very nearly linearly separable in this feature space and a hyperplane is the right hypothesis class. Train–test gap of 0.004 with CV sd 0.0066 means it is not overfitting and has capacity to spare. Its 0.5-point lead over the RBF SVM is around 0.7 of a CV standard deviation, so "logistic regression and SVM are tied at the top, ahead of the forest" is the honest reading.
Decision Tree	Trains to 0.9634 and tests at 0.8985 — a 0.065 gap that is pure variance. A single tree partitions with axis-parallel cuts, so it approximates a diagonal boundary with a staircase and burns depth doing it; `min_samples_leaf=5` bounds this but does not remove it. Notably its CV sd is the *lowest* in the table (0.0021) — it is consistently mediocre, not erratic, so the gap is bias-toward-the-training-set rather than fold sensitivity.
kNN	Train accuracy is exactly 1.0000, which is the `weights="distance"` giveaway: a training point sits at distance 0 from itself and takes infinite weight, so the model reproduces the training set by construction. Real generalisation is 0.8893 — the largest gap in the table (0.111). With 48 scaled numeric dimensions, Euclidean distance is already thinning out (concentration of distances), so a fixed *k* averages over

ML Model Name	Observation about model performance
	neighbours that are not especially near. It is also the model that most depends on the `StandardScaler`: without it, `market_value`-scale columns would dominate the metric outright.
Naive Bayes (Gaussian)	Clear worst on accuracy (0.7060) yet holds an AUC of 0.9465 — the class *ranking* is good and the *calibration* is bad. That signature is exactly what violated conditional independence produces: `offensive_contribution`, `creativity_score`, `possession_impact` and the underlying pass/shot counts are strongly correlated, so NB multiplies what is effectively the same evidence several times and drives posteriors to overconfident extremes, mislabelling near the boundaries while keeping the ordering roughly right. Recall (0.7713) exceeding precision (0.7197) says it over-predicts the broad classes. Fastest to fit in the table at 0.19 s — the accuracy cost is not worth it here, but the AUC shows the features themselves are informative.
Random Forest (Ensemble)	Bagging does what it says: against the single tree, train accuracy rises 0.9634 → 0.9790 while the train–test gap *halves*, 0.065 → 0.035, and test accuracy climbs 0.8985 → 0.9441. Averaging 200 decorrelated trees cancels the variance of any one of them. It still trails logistic regression, which is the useful negative result — the ensemble's extra capacity is spent modelling a boundary that was close to linear to begin with, so there was nothing there to buy.
SVM (RBF)	Statistically tied with logistic regression (0.9533 vs 0.9580, under 1 CV sd apart) at roughly 11× the fit cost, 15.72 s vs 1.44 s. That near-equality is itself the evidence: if the RBF kernel's non-linear boundary bought real separation, it would show as a clear win. It does not, which independently confirms the linear-separability read. Slowest model in the table — `probability=True` adds an internal 5-fold Platt calibration on top of an already superlinear fit.
Overall Winner for your dataset?	**Logistic Regression**, selected on **MCC (0.9416)**. With a 4-class split running from 34.6% to 11.5%, plain accuracy is dominated by Defender and Midfielder and would let a model that ignores Goalkeepers look respectable; MCC accounts for every cell of the 4×4 confusion matrix and does not reward that. Logistic regression happens to top all six metrics, so the choice is not contested here — but it also wins on the criteria that matter operationally: it fits ~11× faster than the SVM it is tied with, produces the smallest artifact (11 KB against the forest's 19 MB), and is the only model whose coefficients can be read directly as "which behaviours mark a Defender".

Streamlit app features

Requirement	Where
Dataset upload option (CSV)	Sidebar → *1 · Test data*
Model selection dropdown	Sidebar → *2 · Model*
Display of evaluation metrics	*Metrics* tab — all six
Confusion matrix / classification report	*Confusion matrix & report* tab
All models on the test data	*Compare all models* tab

Verification

`scripts/verify.py` runs 28 checks and exits non-zero on any failure:

- **Leakage** — train and test player sets are disjoint (936 / 312, 0 shared) and together account for all 1,248 players; the grouping column is not a feature; the grouped split is not more optimistic than a random one.
- **Artifacts** — every pipeline in `metadata.json` loads and predicts, which is what `app.py` does at startup. Catches a version-mismatched pickle before deploy.
- **Robustness** — an unseen categorical level at inference encodes to zeros rather than raising.
- **Deployability** — no artifact over 50 MB (GitHub's warning threshold), 25.9 MB total.
- **App rendering** — `streamlit.testing.v1.AppTest` executes `app.py` against the *pinned* Streamlit and selects each of the six models in turn.

That last group caught two deploy-breaking bugs that artifact-loading tests alone would have missed, because both are Streamlit-API issues rather than model issues:

1. `pandas.DataFrame.style` is an optional accessor gated on `jinja2`, which Streamlit does not pull in transitively. Now pinned, and the call site degrades to an unstyled table rather than taking down the tab.
2. `st.dataframe(..., width="stretch")` is valid from Streamlit 1.49 but raises `TypeError: 'str' object cannot be interpreted as an integer` on the pinned 1.41.1. The app would have booted cleanly and then crashed the instant a grader opened the *Confusion matrix* tab. Replaced with `use_container_width=True`, which is the correct API for the pinned version.

The general lesson: pinning a dependency and then writing code against a *newer* version of its API fails at runtime, not at install time, so the check has to actually execute the app under the pin.

Limitations

- **The dataset is synthetic**, and it shows. Zero missing values across 54,600 × 75 is not a property real match data has, and the `match_result` columns are provably independent of everything else (RF 0.380 vs 0.369 baseline). Conclusions here are about *model behaviour on this feature geometry*, not about football.
- **No hyperparameter search.** These are near-default configurations. A tuned tree or a tuned k would narrow some of the gaps above, and `min_samples_leaf=5` on the forest was chosen for artifact size, not accuracy.
- **Single grouped split plus 5-fold CV**, not repeated CV — so differences under roughly 0.01 accuracy should be treated as noise, which is precisely why the logistic-regression-vs-SVM result is reported as a tie.
- **Position is an easy target.** Goalkeepers are nearly separable on `saves` alone. Almost all the remaining difficulty sits in one cell pair: of the winning model's 69 errors on `test_data.csv`, **46 are Midfielder↔Forward** — a genuinely fuzzy boundary, since an attacking midfielder and a withdrawn forward produce similar match profiles. The per-class rows of the classification report in the app carry more information than the macro averages above.
- **`test_data.csv` is downsampled to 2,000 rows** to stay within Streamlit Cloud's memory quota, so app-reported metrics differ slightly from the table above (computed on the full 3,813-row test split).